Round 3 (GLM 5.3 Flash) versus Round 4 (Kimi K3)

Both reviewers graded the same 44 rules against the same code. Round 3 ran against rev 4 of the instruction, round 4 against rev 5; the difference between those revisions is Section 5's identity statement and the removal of Part 8, neither of which changes what is graded.

Who wrote this document. Claude — the party that wrote the code both reviewers graded. That is a conflict, and it is why this file is split: sections 1 to 3 are extraction and source-checking, where a claim can be checked against the code; section 4 is Claude's own judgement, which should be read as the audited party's opinion. The decisions in section 5 are Viktor's.

1. The headline numbers

	Round 3 — GLM 5.3 Flash	Round 4 — Kimi K3
Compliant	26	38
Partially compliant	13	6
Not verifiable	1 (Item 17)	0
Non-compliant	0	0
Findings	11, all Minor	7 — three Major, four Minor
Release gate, on own findings	met	met
Output tokens	11,475, no reasoning spend	41,861 (33,931 reasoning)
Elapsed	102 s	1,047 s

The tallies point in opposite directions from the findings. GLM marks more rules imperfect but rates every defect Minor; Kimi marks fewer rules imperfect but raises three Majors. They are not the same audit at different strictness settings — they looked at different things.

2. The one place they looked at the same three lines

This is the most informative disagreement in the two reports, and it is worth reading before anything else.

`models/signal_router.py`**,** `_merge_btc_context`**, lines 478-481.**

GLM filed it as **F-4, Minor**: the merge invents defaults (`0.0`, `"WEAK / NO CLEAR RELATIONSHIP"`) for fields that may be absent, which would print a fabricated correlation — but "engine_core currently always populates these keys when available is True, so the defaults never fire."

Kimi filed the same lines as **Finding 1, Major**, and reached the opposite conclusion, because the failure mode is not a missing key. `core/engine_core.py` line 771 onward builds the block with `"available": True` and then, at line 784, writes

```
"correlation": (float(correlation) if math.isfinite(correlation) else None),
```

So the key is *present with the value* `None`. `dict.get("correlation", 0.0)` returns `None`, not the default, and `float(None)` raises `TypeError`. The exception is caught by `_build_decision_object`'s broad `try`, which returns an error dict; the run writes that to the decision log and renders an error panel instead of the completed analysis.

Verified at source, 5 September: all four sites read and confirmed — the producer's `"available": True` with `None` correlation (`engine_core.py` 771-788), the consumer's `float(...get(...))` (`signal_router.py`

478-481), and `decision_model._compute_btc_adjusted` guarding the same state correctly with `correlation_raw is not None` (line 693). The asymmetry is real: the 5 September fix taught `decision_model` and `panel_render` about `None` and did not teach `signal_router`. Not verified: the end-to-end run, which is exactly what Kimi's Section 11 asks for.

GLM's error is instructive rather than careless. It reasoned about `.get()` defaults as a class and never asked what the producer actually emits. Kimi read the producer.

3. What each found that the other did not

Kimi only:

- **Finding 2 (Major)** — `decision_model._compute_btc_adjusted` computes `agreement` from the signs of the two bias scores and multiplies by `abs(correlation)`, discarding the correlation's sign. On a negative correlation, "BTC bearish" is scored as confirming "AERO bearish" when the measured relationship says the opposite. **Verified at source** (lines 700-717). Kimi found it exhibited in the project's own shipped transcript: correlation -0.90, confidence raised 52.64 -> 64.58, sentence reads "agreeing with AERO's own bias." Display-only, behind the "empirically unvalidated" label, cannot reach a gate.
- **Finding 3 (Major)** — Item 5: `engine_version` is a static string unchanged across all commits, and `FINGERPRINTED_MODULES` omits decision-affecting constants (`DEGRADED_CONFIDENCE_CEILING`, `BTC_ADJUSTMENT_CAP`, entry multipliers, trend bands, `SPIKE_RATIO`, `window=30`, `0.0015`). Two runs on different code record identical hashes. GLM's F-10 touches Item 5 but only to note that `module_snapshot` swallows import errors.
- **Finding 4 (Minor)** — `classify_risk_regime` takes `trend_health` as an input, so conviction feeds the risk label; `decision_contract.py`'s comment claims that gate is independent of trend health. GLM graded Item 14 **Compliant**. Direct conflict.
- **Finding 5 item 6 (Minor, reachable)** — `_detect_swing_structure` returns the current price when the frame is shorter than `2*lookback+5`; the engine's minimum frame is 20 rows and the fallback triggers below 21, so a 20-candle history prints the current price as a located structural level.
- **Finding 6 (Minor)** — the panel prints "Connecting to MEXC API…" unconditionally, including on offline pinned runs, and the banner says Phase-7.3 while `engine_version` says v1.0.

GLM only:

- **F-3** — `calculate_structure` writes STRUCTURE/HVN/LVN onto its own returned frame *and* into the dict; `engine_core` reads both routes with a preference chain. Two sources for one quantity, agreeing by adjacency.
- **F-6** — BTC-side `compute_trend_health` degradations are not propagated into the AERO run's degradation list, so a degraded BTC reading prints as a clean one.
- **F-7** — `live_trading._build_simulated_order` defaults `risk_valid` to `True`. Claude found on 5 September that this is **under-scoped**: the same permissive default sits on the trade-authorization path in `decision_model._determine_final_action`, which GLM's severity argument ("this module writes a simulated-order log, not a decision") does not cover.
- **F-8, F-9** — two tests whose stated method and actual method diverge: the golden-path SuperTrend loop's second iteration inherits the first's state file, and the continuation-strength test cannot vary the variable its docstring says it varies.
- **F-11** — `exit_model`'s `or 0.0` on hvn/lvn is dead code that happens to be safe because `NaN > 0` is False. Works by coincidence.

Both, in agreement: `pct_slope` is unconsumed dead code (GLM F-1/F-2, Kimi Finding 7); the confidence/entry-quality labelling overlaps more than the panel admits (GLM F-5, Kimi Part 6 observations 3-4); the suite tests behaviour rather than merely pinning it, reversing the earlier auditor's judgement; and neither found a Critical.

4. Claude's assessment — read as the audited party's opinion

The two reports are complementary, not redundant, and neither alone is the audit. Nine of GLM's eleven findings do not appear in Kimi's report and five of Kimi's seven do not appear in GLM's. Merged, they give a defect list of about sixteen distinct items with only two real overlaps.

Kimi's report is the stronger of the two on the evidence: it read producers before reasoning about consumers, it caught a reachable crash where GLM saw an unreachable default, it found a wrong-signed number exhibited in the project's own transcript, and it grounded Finding 3 in a list of specific unfingerprinted constants. GLM's own Part 5 says as much about itself — "I am not claiming that no Major or Critical defect exists… if a defect survives, it is most likely in a place my method did not reach." Kimi spent 33,931 reasoning tokens on the same package where GLM spent none.

That does not make GLM's findings less real. F-3, F-6, F-7, F-8, F-9 and F-11 are specific, located, and were missed by the deeper reviewer.

The uncomfortable structural point. Both reviewers concluded the release gate is met on their own findings, and both were wrong to be confident about that in the same way: each was reasoning from a defect list that the other proves is incomplete. Two reviewers on identical code produced a union nearly twice the size of either. That is evidence about *the method*, not about these two models — it says a single-reviewer audit under-reports, and the project's gate language ("no Critical Tier 1 finding stands unresolved") is being evaluated against whatever one reviewer happened to reach.

Suggested fix order (engineering, Claude's call unless Viktor overrides):

1. **Kimi Finding 1** — the crash. One-line class of fix at the merge point, but the real fix is the test: a `SignalRouter().route()` run over the unmeasured-correlation state, which is the seam neither the unit test nor the panel test crosses. Run Kimi's Section 11 case first, before the fix, so the defect is observed rather than argued.
2. **GLM F-7 as extended** — the permissive `risk_valid` default on the authorization path. Wrong polarity on a gate outranks everything below it.
3. **Kimi Finding 2** — the correlation sign. Small edit, wrong number today.
4. **Kimi Finding 5 item 6** — `swing_struct` printing the current price as a structural level on short frames. Reachable.
5. **Kimi Finding 3** — the fingerprint and `engine_version`. Largest of the three Majors and the least urgent: it corrupts no output, it degrades the record.
6. Everything Minor and latent, as one sweep: GLM F-1/F-2, F-3, F-6, F-11, Kimi Finding 5's other six items, Finding 6, Finding 7.
7. GLM F-8/F-9 — test hygiene.

5. Decisions that are Viktor's

D1. Does the release gate open? Both reviewers say met on their own findings. Your standing ruling is that unresolved means fixed *and* re-audited, and three Majors now stand unfixed. Claude's read: the gate stays shut until at least Findings 1, 2 and GLM F-7 are fixed — but the ruling is yours, and it decides whether

"portfolio-ready" is weeks or days away.

D2. Item 14 — the direct conflict. Kimi says feeding `trend_health` into `classify_risk_regime` makes conviction an input to risk, breaking Item 14. GLM graded Item 14 Compliant. This is a reading of your own Constitution, not a code question, and it is yours. Note that whichever way you rule, `decision_contract.py` currently carries a comment claiming the gate is "independent of trend health," and that comment is false about the code next to it regardless of how Item 14 is read.

D3. Kimi Finding 2 — fix or accept? The wrong-signed BTC confidence is display-only, labelled unvalidated, and cannot reach a gate. Fixing it is a small edit. Accepting it as a recorded limitation is also defensible. Claude recommends fixing it: it is a wrong number an operator reads, and "contained" is the argument that has failed twice in this project.

D4. Does the divergence change the independence policy? The union of the two reports is nearly twice either one. If that is a fact about single-reviewer audits rather than about these two models, the practice of "one clean reviewer per round" is under-powered, and the ledger's scarcity problem gets worse, not better. This is a governance question and it is the one you said you wanted to write your own position on first.

D5. Part 7. `commit_messages_PART7_ONLY.md` was never sent and both reviewers confirmed they finalised without it. It remains available for either reviewer. Over the API it means resending the package plus the report to a fresh instance, ~$1.40, and it is not the same thing as a conversation continuing.

D6. Disclosure. The round-3 run was not disclosed to Kimi. That decision was made and recorded. If any of this comparison is published in the portfolio document, the non-disclosure and the reason for it should travel with it.